# WireGuard VPN with IdP-Driven Multi-Factor and Risk-Based Authentication

Ana Rita Silva
*MSc. in Cybersecurity*
*DETI, University of Aveiro*
Aveiro, Portugal
aritafs@ua.pt (NMEC 114220)

Carlos Coelho
*MSc. in Cybersecurity*
*DETI, University of Aveiro*
Aveiro, Portugal
carlos.coelho@ua.pt (NMEC 43483)

Miguel Reis
*MSc. in Cybersecurity*
*DETI, University of Aveiro*
Aveiro, Portugal
mreis@ua.pt (NMEC 108545)

Paulo Pinto
*MSc. in Cybersecurity*
*DETI, University of Aveiro*
Aveiro, Portugal
paulojnpinto02@ua.pt (NMEC 103234)

***Abstract*—WireGuard has no built-in user authentication: every peer must be pre-configured with static keys and a fixed tunnel IP. We present a login layer that provisions all per-session WireGuard parameters dynamically after MFA+RBA authentication through an OAuth 2.0 IdP orchestrated by a custom C PAM module. The auth channel is a hand-implemented Noise-IK tunnel keyed on the gateway's existing WireGuard public key, so no additional PKI is needed. The system supports three factors (password challenge-response, TOTP, and FIDO2 with user verification required), selected at runtime by a five-signal risk engine, and ships with five additional hardening measures.**

***Index Terms*—WireGuard, VPN, Multi-Factor Authentication, Risk-Based Authentication, Noise-IK, OAuth 2.0, FIDO2, TOTP, PAM, Identity Provider**

## I. Introduction

This report documents our group project for the *Identity and Access Management* (IAA) course at the University of Aveiro. The project brief asks us to design and implement *"a secure user authentication protocol for WireGuard VPN"* [1] together with the supporting client and server applications, MFA, RBA, an IdP, a custom PAM module, and a helpdesk enrolment interface. This report describes what we built, the design decisions behind each component, and where in the codebase each project requirement is satisfied.

### A. Problem Statement

WireGuard tunnels packets between peers that already know each other's Curve25519 public key, private tunnel IP, and pre-shared key. For a host-to-gateway deployment serving many users, the gateway must pre-configure all six per-peer parameters before any client can connect, and clients must avoid private IP collisions out-of-band. We solve this by building a login layer on top of WireGuard: the user authenticates through a browser-based MFA flow gated by a risk engine, and only on success does the gateway generate and hand over the per-session WireGuard parameters so the tunnel can come up. The entire stack, login client, gateway daemon, identity provider, PAM module, and helpdesk, runs from a single `docker compose up`, with the gateway's WireGuard public key as the only trust anchor.

### B. Mapping to Project Requirements

Table I maps every mandatory project requirement to the section of this report and the file in the codebase that implements it.

TABLE I
PROJECT REQUIREMENTS VS. WHERE THEY ARE IMPLEMENTED.

| Requirement | Section | Code location |
|---|---|---|
| UserAuthn application | III | `client/userauthn.py` |
| SrvAuthn application | III | `server/srvauthn.py` |
| Six WG parameters per side | III-E | `server/srvauthn.py`, `common/db.py` |
| WG public key as source of trust | II-B | `common/noise.py` |
| IdP (OAuth 2.0) | III | `idp/app.py` |
| HTTP-over-secure-tunnel to IdP | III-D | `client/proxy.py`, `server/proxy.py` |
| OAuth Device Authorization Grant | III | `idp/app.py` |
| MFA: password (challenge-response) | IV-B | `idp/app.py`, `idp/static/js/` |
| MFA: TOTP | IV-C | `idp/app.py`, `common/db.py` |
| MFA: FIDO2 | IV-D | `idp/app.py` |
| RBA (IP, ASN, failures, time) | VI | `idp/rba.py` |
| PAM module orchestrating MFA+RBA | V | `idp/pam_iaa/src/pam_vpnmfa.c` |
| Helpdesk enrolment interface | IV-A | `idp/helpdesk/app.py` |

### C. Report Structure

Section II gives the system architecture and trust model. Section III walks through the authentication protocol end-to-end and shows how the six WireGuard parameters per side are assigned. Section IV covers the three MFA factors and the helpdesk enrolment interface. Section V describes the custom C PAM module. Section VI describes the RBA engine. Section VII covers the additional security hardening measures

we implemented on top of the core functionality. Section VIII demonstrates that the system works end-to-end. Section IX concludes and notes what is intentionally not implemented.

## II. Architecture

### A. Five Processes, Two Hosts

The system is composed of five processes spread across two hosts (Fig. 1):

- **UserAuthn** (`client/userauthn.py`): runs on the user host. Generates the PSK locally (parameter 6 of the WG config), opens the Noise-IK auth tunnel, drives the OAuth Device Grant, applies the WG config.
- **SrvAuthn** (`server/srvauthn.py`): runs on the gateway host with `network_mode: host` so it can manage the in-kernel `wg0` interface. Noise-IK responder, talks to the IdP, generates each client's keypair, leases IPs, and registers peers.
- **IdP** (`idp/app.py`): Flask OAuth 2.0 server implementing RFC 8628 [2]. Hosts the browser MFA pages, calls `pam_authenticate()` via our C library.
- **pam_vpnmfa.so** (`idp/pam_iaa/src/pam_vpnmfa.c`): the custom PAM module loaded by the IdP through the service profile `/etc/pam.d/iaa-vpn`. Decides which factors are needed and in what order (Section V).
- **Helpdesk** (`idp/helpdesk/app.py`): a loopback-only Flask app for enrolment, bound to `127.0.0.1:8081` per the project's instruction that the interface has no security-related validations (Section IV-A).

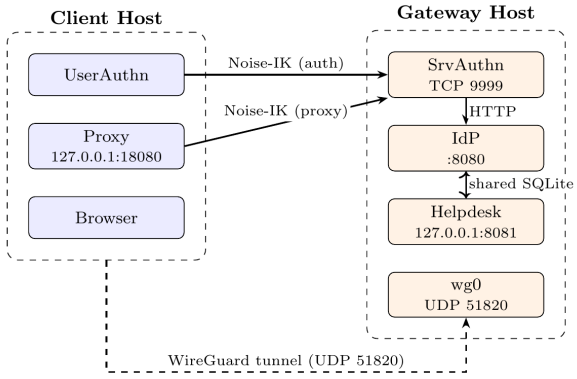



Fig. 1. Two-host deployment. Solid arrows show the authentication-phase connections; the dashed arrow is the WireGuard data-plane tunnel that forms after authentication.

### B. Single Root of Trust

The gateway's 32-byte Curve25519 WireGuard public key is the *only* trust anchor a client needs. It is distributed once (in `base.conf`), and everything else follows from it.

That single key plays two roles:

1) **Noise-IK responder static key.** `SrvAuthn` loads `/etc/wireguard/privatekey-server` at startup and uses it as the Noise-IK static private key. The handshake fails with an AEAD-tag mismatch if a man-in-the-middle does not possess the matching private half.
2) **WireGuard data-plane key.** The same private key signs Noise-IK handshakes for the data plane once `wg-quick up` runs on the client.

The browser side is protected by the same anchor through the local HTTP proxy (Section III-D), so the user never has to trust a TLS certificate.

### C. Deployment and Storage

The entire stack launches with a single `docker compose up`. Table II lists the four containers: three permanent server-side processes plus an optional client used for single-machine demos.

TABLE II
Docker containers and their network configuration.

| Container | Role | Port(s) |
|---|---|---|
| `vpn-idp` | IdP, OAuth, MFA pages | 8080 |
| `vpn-helpdesk` | Enrolment UI (host net) | 127.0.0.1:8081 |
| `vpn-srvauthn` | Auth daemon + `wg0` (host net) | 9999, 51820/udp |
| `vpn-client` | Optional demo client | 18080 |

Because the three server-side processes are isolated containers, they cannot share in-memory state. All persistent data (user accounts, enrolled MFA credentials, active OAuth sessions, RBA history, and WireGuard peer assignments) lives in a single SQLite database (`/data/vpnauth.db`) that is bind-mounted into every container that needs it. WAL mode allows the IdP and helpdesk to read concurrently without blocking writes from `SrvAuthn`, and the few operations that read-then-write atomically (IP lease, nonce consumption, token redemption) use `BEGIN IMMEDIATE` to prevent races under concurrent logins.

## III. Authentication Protocol

This section describes the protocol end-to-end, then shows how each WireGuard parameter is produced and by which component.

### A. The Auth Tunnel (Noise-IK)

All HTTP traffic between `UserAuthn` and the IdP passes through a hand-built secure tunnel so the client does not need to trust a separate TLS certificate. We chose Noise-IK over TLS specifically because Noise-IK lets us reuse the gateway's WireGuard public key as the responder identity, collapsing the auth tunnel's trust anchor and the VPN data-plane trust anchor into one.

`common/noise.py` is a from-scratch implementation of the Noise-IK handshake (Noise Protocol Framework [3], revision 34) over `cryptography.hazmat` primitives only: X25519 [4], ChaCha20-Poly1305 [5], BLAKE2s [6]. No third-party Noise library was used.

The "IK" in the name means the *I*nitiator already *K*nows the responder's static *K*ey before the handshake begins, which

is precisely our situation, since the client already holds the gateway's WireGuard public key from `base.conf`. The pattern therefore maps directly onto our trust model with no extra key distribution.

The IK pattern is a two-message handshake: in message 1 the initiator sends its ephemeral and (encrypted) static public keys plus an encrypted payload; in message 2 the responder replies with its ephemeral and an encrypted payload, completing the four Diffie-Hellman operations $es$, $ss$, $ee$, $se$. Each operation mixes one pair of keys (ephemeral or static, initiator or responder) into the running shared secret, so that by the end both the fresh session and the long-term identities of *both* parties are authenticated. After the handshake, both sides hold a pair of ChaCha20-Poly1305 `CipherState` objects (one per direction), and frames are exchanged with a 4-byte big-endian length prefix (`common/framing.py`).

### B. End-to-End Flow

The complete protocol proceeds in nine steps (Fig. 23 in Appendix A):

1) `UserAuthn` generates a fresh 32-byte PSK via `wg genpsk` on the client side.
2) `UserAuthn` opens a TCP connection to `SrvAuthn` on port 9999 and runs the Noise-IK handshake. The responder static key is the server's WireGuard public key from the client's `base.conf`.
3) Over the encrypted tunnel, `UserAuthn` sends a JSON hello containing service tag (`"auth"`), username, PSK, and target WireGuard interface name. `SrvAuthn` validates the PSK shape (32-byte base64) before any further processing.
4) `SrvAuthn` calls `POST /oauth/device/code` at the IdP, attaching the username as a login hint and the client's source IP for the RBA engine.
5) `SrvAuthn` returns {`user_code`, `verification_uri`} to the client over the tunnel; `UserAuthn` opens the URI in the default browser.
6) The browser reaches the IdP through the local HTTP proxy (Section III-D). The user submits the device code; the IdP starts the PAM-orchestrated MFA sequence (Section V).
7) `SrvAuthn` polls `POST /oauth/token` every five seconds until the IdP returns an access token.
8) `SrvAuthn` calls `GET /oauth/userinfo` with the bearer token (single-use; see section VII) and receives the authenticated username.
9) `SrvAuthn` generates a fresh WG keypair (`wg genkey | wg pubkey`), leases the next free IP from `wg_peers` inside a `BEGIN IMMEDIATE` transaction, registers the new peer with `wg set wg0 peer ...`, and sends the full WG config back over the tunnel. `UserAuthn` writes `/etc/wireguard/wg0.conf` and runs `wg-quick up wg0`.

### C. Why OAuth Device Authorization Grant

`UserAuthn` is a terminal application and cannot expose an HTTP redirect URI, so a standard authorization-code flow is not an option. The Device Authorization Grant (RFC 8628 [2]) solves this by having the terminal poll for a token while the user completes the flow in a separate browser window, keeping the CLI and the browser completely decoupled.

### D. The Local HTTP Proxy

To keep the browser flow on the same trust anchor as the auth tunnel, `UserAuthn` listens on `127.0.0.1:18080` (`client/proxy.py`); every browser connection opens a fresh Noise-IK tunnel to `SrvAuthn` with the first frame {`"service":"proxy"`}, and `server/proxy.py` pumps bytes between the tunnel and the IdP at `localhost:8080`.

The IdP's `ORIGIN` is set to `http://localhost:18080`, so every URI the IdP generates (including the OAuth `verification_uri_complete`) points at the proxy. The browser's effective trust anchor is therefore the same WireGuard public key, just expressed through a different pipe.

WebAuthn's `RP_ID` is `localhost`, which covers both the proxy origin (`http://localhost:18080`) and the helpdesk's `http://localhost:8081`, so security-key credentials registered via the helpdesk validate at login without re-enrolment. Using `localhost` rather than the numeric `127.0.0.1` is required: browsers treat `localhost` as a secure context for WebAuthn purposes, but do not consistently extend that treatment to the IP literal.

### E. WireGuard Parameter Assignment

A WireGuard peer requires six configuration values per side (client and server). Tables III and IV show which component produces each one and how.

TABLE III
WIREGUARD PARAMETERS WRITTEN TO THE CLIENT

| Parameter | How produced |
|---|---|
| 1. Client secret key | `wg genkey` on the server, shipped over Noise-IK |
| 2. Client tunnel IP | `db.lease_next_ip` (atomic) on the server |
| 3. Server public key | Pre-installed in `base.conf` |
| 4. Server public IP | Pre-installed in `base.conf` |
| 5. Server UDP port | Pre-installed in `base.conf` |
| 6. Pre-shared key | `wg genpsk` on the client, sent in the hello |

TABLE IV
WIREGUARD PARAMETERS APPLIED ON THE SERVER

| Parameter | How produced |
|---|---|
| 1. Server secret key | `/etc/wireguard/privatekey-server` (generated once at setup) |
| 2. Server tunnel IP | `wg0.conf` `Address = 10.10.10.1/24` (written by `scripts/init-wg.sh`) |
| 3. Server UDP port | `srvauthn.conf` (`wg_listen_port`, default 51820) |
| 4. Client public key | `wg pubkey` on the server, then `wg set` |
| 5. Client allowed IP | Same lease as client param 2 |
| 6. Pre-shared key | Echo of client-supplied PSK |

The PSK (client parameter 6) is generated locally by `UserAuthn` before the tunnel opens; the keypair and leased IP (client parameters 1 and 2) are created by `SrvAuthn` after authentication; the server's public key, public IP, and listen port (client parameters 3–5) are pre-distributed in `base.conf` and serve as the client's only required prior knowledge.

One deliberate choice concerns the server tunnel IP (server parameter 2): rather than an explicit field in `srvauthn.conf`, the server's address (`10.10.10.1/24`) is written directly into `wg0.conf` by `scripts/init-wg.sh` (the one-time key/config bootstrap invoked by `scripts/setup.py`), following standard `wg-quick` convention. `srvauthn.conf` carries `internal_subnet = 10.10.10.0/24`, which `SrvAuthn` uses only for client IP lease allocation (starting from `.2`); the server permanently holds `.1`.

### F. Lease Lifecycle

Every row in `wg_peers` carries an `expires_at` (default 24 h, configurable in `srvauthn.conf`). A daemon thread in `server/leases.py` polls every 60 seconds, finds active rows with `expires_at < now`, runs `wg set wg0 peer <pub> remove`, and marks the row inactive. Provisioning is the only insert path; the sweeper is the only delete path.

## IV. MULTI-FACTOR AUTHENTICATION

We implemented three authentication factors in order of increasing robustness: a password challenge-response scheme, TOTP, and FIDO2/WebAuthn. Which factors are required for a given login depends on the RBA score (Section V).

### A. Helpdesk Enrolment

Credential management is handled by a separate Flask app, `idp/helpdesk/app.py`, bound to `127.0.0.1:8081` with `network_mode: host`. Being bound to the loopback interface, it is physically unreachable from outside the gateway machine. Its access boundary is enforced by the network, not by authentication logic.

The helpdesk shares the same SQLite database as the IdP through a bind-mount, so credentials registered here become immediately usable at login. It exposes endpoints for all three factors (`/user/<u>/password`, `/totp`, `/fido2/start`, `/fido2/finish`) plus credential listing and removal.

### B. Password (Challenge-Response)

The raw password never leaves the browser.

**Enrolment:**

The helpdesk derives a 32-byte verifier with scrypt [7], [8]:

$$v = \text{scrypt}(pw,\ s,\ N = 2^{14},\ r = 8,\ p = 1,\ \text{dkLen} = 32)$$

where $pw$ is the user's password and $s$ is a 16-byte random salt; only $(s, v)$ are stored in the `passwords` table.

**Login:**

When the PAM module requests the password factor, the browser fetches `/oauth/mfa/<sid>/password/challenge`, which returns the salt, a fresh 32-byte nonce, and the scrypt parameters. A pure-JavaScript scrypt implementation (`idp/static/js/scrypt.js`) recomputes $v'$ in the browser and submits

$$\text{proof} = \text{HMAC-SHA256}(v',\ \text{nonce})$$

The server compares against `HMAC-SHA256(v, nonce)` with `hmac.compare_digest`. The nonce is burned atomically (`BEGIN IMMEDIATE`) before the comparison, so a replay finds the nonce already consumed.

The practical effect is twofold. Because only the proof crosses the network, an eavesdropper (or the IdP itself) never sees the password and cannot reuse a captured proof, since each nonce is single-use. And because the database stores only the scrypt verifier $v$, a database leak yields neither the password nor a value that authenticates on its own: an attacker would still have to brute-force the scrypt hash. Fig. 2 shows the browser form during this step.

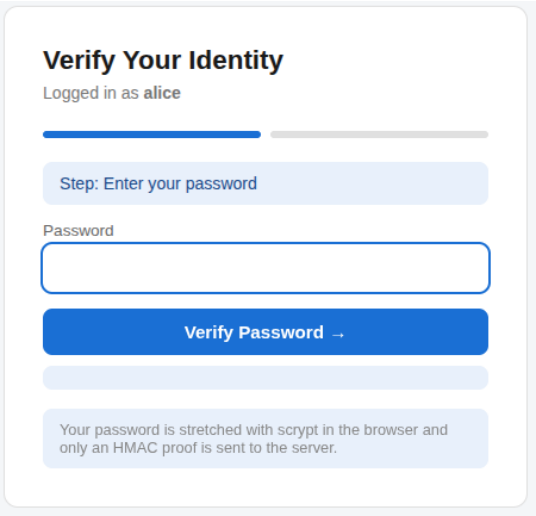



Fig. 2. Password step: scrypt and the HMAC proof run client-side.

### C. TOTP

RFC 6238 [9], six digits, 30-second step, ±1 window, via `pyotp.TOTP.verify(code, valid_window=1)`.

Enrolment happens at the helpdesk: `POST /user/<u>/totp` generates a fresh secret and shows

it as a QR code (Fig. 3). Confirmation requires the user to submit a valid code, after which the row in `totp_secrets` flips to `confirmed=1`.

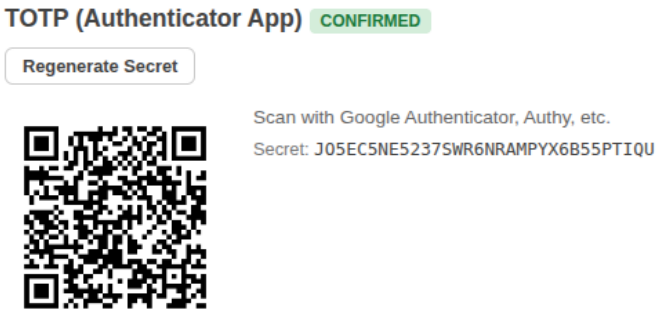



Fig. 3. TOTP QR code during helpdesk enrolment.

The secret is stored encrypted at rest under AES-256-GCM (`common/db.py:_encrypt`); see Section VII. At login the browser presents a 6-digit code entry form (Fig. 4).

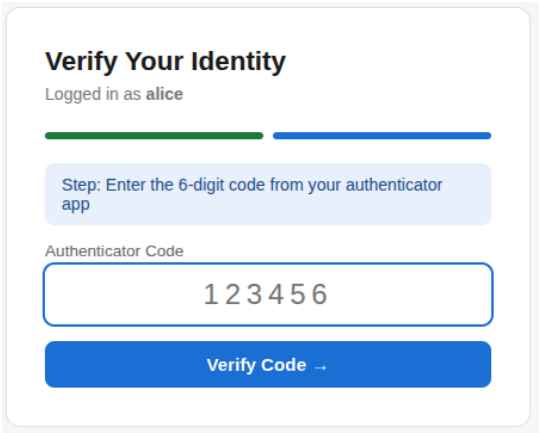



Fig. 4. TOTP step during VPN authentication.

### D. FIDO2 / WebAuthn

We use the `webauthn` (v≥2) Python library on the server and the browser's native `navigator.credentials` API on the client.

Enrolment uses the helpdesk's `/fido2/start` and `/fido2/finish` endpoints. The browser's WebAuthn dialog first asks the user to select the authenticator device (Fig. 5).

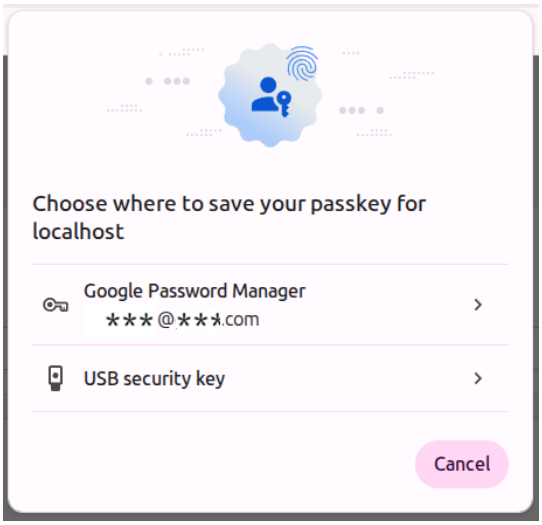



Fig. 5. Authenticator selection during FIDO2 registration.

Both registration and authentication set `UserVerificationRequirement.REQUIRED`, which forces the authenticator to validate a local PIN, biometric, or password before releasing the credential's private key. The browser therefore prompts for the key's PIN (Fig. 6). A physically stolen, unlocked security key cannot satisfy this factor because the assertion will be missing the `uv` flag and the server rejects it.

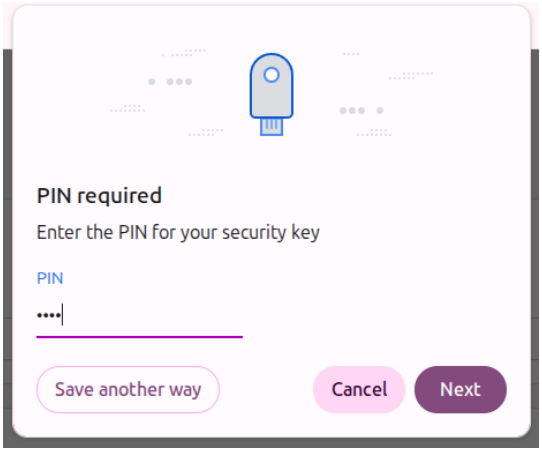



Fig. 6. PIN prompt (user verification) during FIDO2 registration.

On success the credential appears in the user's helpdesk panel (Fig. 7).

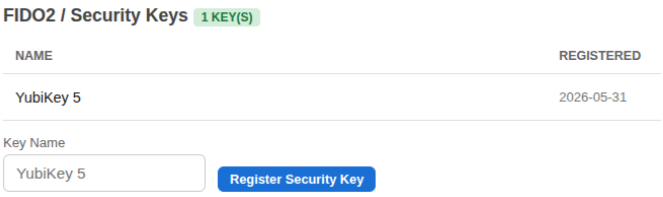



Fig. 7. Credential listed after successful FIDO2 registration.

At login the IdP issues a challenge via `/oauth/mfa/<sid>/fido2/start` (bound to the Flask session), the browser-side glue (`idp/static/js/webauthn.js`) calls `navigator.credentials.get()`, and the assertion is verified with `verify_authentication_response(..., require_user_verification=True)`. The credential's sign counter is updated on every successful assertion to detect cloned authenticators.

## V. PAM Orchestration

MFA and RBA orchestration runs through PAM: a custom C module (`pam_vpnmfa.so`) decides which factors are required and in what order, while a Python bridge in `common/libpam.py` drives `pam_authenticate` from the Flask IdP through `ctypes`. This keeps the policy (what is required) strictly separated from the mechanism (how each factor is verified).

### A. The Custom PAM Module

`idp/pam_iaa/src/pam_vpnmfa.c` compiles to a standard PAM module shared object:

```
gcc -Wall -Wextra -fPIC -shared -O2 \
    -o pam_vpnmfa.so pam_vpnmfa.c -lpam
```

It is installed in the system PAM module directory. The service profile `/etc/pam.d/iaa-vpn` wires our module into the `auth` stack, with `pam_permit` satisfying the unused `account` phase:

```
auth    required pam_vpnmfa.so
account required pam_permit.so
```

When the IdP calls `pam_authenticate("iaa-vpn", user)`, the module:

1) Reads `PAM_USER` and `PAM_RHOST` from the item store.
2) Reads the environment variable `IAA_RISK_SCORE` (an integer 0–100) that the Python bridge sets before the call.
3) Maps the score to a tier: scores below 30 require only password; 30–59 require password and TOTP; 60 and above require all three factors (password, TOTP, FIDO2). The module emits a `PAM_TEXT_INFO` message such as `"INFO:tier=L2 score=35"` for the bridge to log.
4) Walks the required factor list in the fixed order `[password, totp, fido2]`, sending one `PAM_PROMPT_ECHO_OFF` conversation prompt `"FACTOR:<name>"` per factor.
5) Interprets the response: `"OK"` advances; `"UNAVAILABLE"` returns `PAM_AUTHINFO_UNAVAIL`; anything else (including `"FAIL"`) returns `PAM_AUTH_ERR`.

Keeping the tier policy in the C module (rather than in Python) means a security audit of "what factors are required" only needs to read one file. The PAM module is the single source of truth for factor selection.

### B. Driving libpam from Flask

The Flask IdP cannot call `pam_authenticate` directly because `libpam` is a C library; we wrote a small `ctypes` wrapper in `common/libpam.py` that exposes `pam_start`, `pam_authenticate`, `pam_end`, and a Python-side conversation callback.

The conversation contract is fixed:

TABLE V
PAM CONVERSATION CONTRACT BETWEEN THE MODULE AND THE PYTHON BRIDGE.

| Prompt | Allowed response |
|---|---|
| `FACTOR:password` | `OK` / `FAIL` / `UNAVAILABLE` |
| `FACTOR:totp` | `OK` / `FAIL` / `UNAVAILABLE` |
| `FACTOR:fido2` | `OK` / `FAIL` / `UNAVAILABLE` |
| `INFO:tier=...` | (any string, treated as ack) |

### C. Threading Model

`pam_authenticate` is blocking but our browser interaction is asynchronous: each factor takes a separate HTTP round-trip. `idp/pam_bridge.py` uses `threading.Event` objects to park the conversation callback until Flask supplies a result:

1) `start_authentication(sid, user, ip, score)` spawns a worker thread that calls `pam_authenticate`.
2) The conv callback parks on `prompt_event` after recording the current `FACTOR:...` prompt.
3) The next Flask route (e.g. `/oauth/mfa/<sid>/totp`) calls `wait_for_prompt(sid)` and renders the relevant factor form.
4) After verifying the factor, the route calls `submit_factor_result(sid, "OK"|"FAIL"|"UNAVAILABLE")`, which unparks the conv callback and the PAM thread proceeds to the next factor or returns.

A 5-minute factor timeout reaps abandoned sessions; Flask handlers use short `wait_for_prompt` timeouts so no HTTP worker is ever blocked on PAM indefinitely.

## VI. Risk-Based Authentication

The RBA engine in `idp/rba.py:compute_risk` evaluates five signals before each authentication attempt and returns a numeric risk score that drives the PAM tier selection (Section V).

TABLE VI
RBA RISK SIGNALS AND THEIR WEIGHTS.

| Signal | Weight |
|---|---|
| First-ever login (no prior IP history) | $+0.30$ |
| New IP address for existing user | $+0.20$ |
| New ASN for existing user | $+0.15$ |
| Recent failures in last 30 min | $+0.10 \times n$, max 0.40 |
| Unusual hour (00:00–06:00 UTC) | $+0.20$ |

### A. Risk Signals

`compute_risk(username, ip, user_agent)` aggregates the five signals in Table VI into a float in $[0, 1]$, then scales it to $[0, 100]$ for the PAM module via `IAA_RISK_SCORE`.

ASN resolution uses a locally mounted db-ip ASN database in MMDB format. No external HTTP calls are made during a login attempt. For private or loopback addresses (the Docker single-machine demo), the ASN lookup is skipped automatically; if the MMDB is missing entirely, only the ASN signal is disabled and the other four continue to work.

The resulting score maps directly to the PAM tier: scores below 30 require password only (L1); 30–59 require password and TOTP (L2); 60 and above require all three factors (L3).

### B. Fail-Closed Policy

If a user is assigned tier L2 or L3 but has not enrolled the required extra factor, `compute_risk` returns `factors=None` and the IdP renders a 403 "contact your helpdesk" page. We deliberately do *not* silently downgrade to a weaker tier: a system that accepts password-only authentication for users without a TOTP token when the risk engine says they should provide one would offer no MFA guarantee under elevated risk.

On successful authentication the client IP and ASN are recorded in `rba_ip_history`, so subsequent logins from the same location attract a lower score. All authentication events (start, per-factor outcome, success/failure) are written to `rba_events`, both for audit and for the recent-failures signal, which is evaluated at the *start* of each new login attempt and therefore reflects failures from previous sessions, not the current one.

## VII. Additional Security Hardening

Beyond the core authentication requirements we implemented five additional hardening measures, each described below.

### A. Token Hygiene (B1)

OAuth access tokens are 48-byte values produced by `secrets.token_urlsafe`. Only their SHA-256 hash is stored; the plaintext appears only in the `POST /oauth/token` response. Tokens have a 60-second TTL and are single-use: `GET /oauth/userinfo` redeems the token with a conditional single-statement update (`UPDATE oauth_tokens SET used_at=now WHERE token_hash=? AND used_at IS NULL`) and accepts the call only when it changed exactly one row. A concurrent or replayed call updates zero rows and returns 401, so a database leak exposes zero usable tokens.

### B. Account Lockout (B2)

Five failed factor verifications within a 5-minute window lock the account for 15 minutes. The lockout check runs at the top of every MFA route, before any cryptographic work, so brute-force attempts are cheap to refuse. Lockout is per-account (not per-IP), so rotating source addresses does not help an attacker. The lock window is not extended by further failures during the locked period since that would let an attacker weaponise the policy into a denial of service against legitimate users. Fig. 8 shows the error page returned on a login attempt against a locked account.

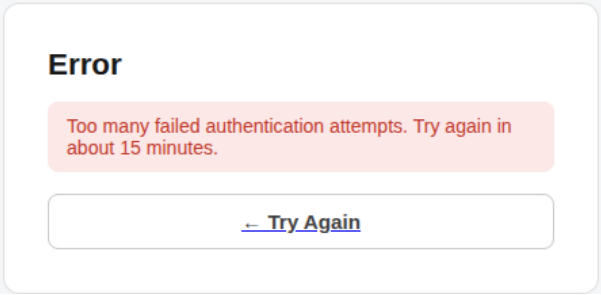



Fig. 8. Account locked after five failed factor verifications.

### C. Encrypted TOTP Secrets at Rest (B3)

TOTP secrets are stored as $\text{nonce}_{12} \,\|\, \text{ciphertext} \,\|\, \text{tag}_{16}$ under AES-256-GCM. The 32-byte master key is loaded from `IAA_MASTER_KEY` and validated at startup; if the variable is missing, empty, or contains a placeholder value, the process exits with code 78.

### D. Peer Lease Expiry (B4)

Every row in `wg_peers` carries an `expires_at` timestamp (default 24 h, configurable in `srvauthn.conf`). A daemon thread in `server/leases.py` polls every 60 seconds, removes expired peers from the `wg0` interface via `wg set wg0 peer <pub> remove`, and marks the row inactive. This limits how long a compromised or abandoned session remains active on the gateway.

### E. FIDO2 User Verification Required (B5)

Both the helpdesk registration and the login flow set `user_verification=REQUIRED` for all WebAuthn operations. The library default is `PREFERRED`, which silently falls back to presence-only verification; `REQUIRED` ensures a PIN or biometric check is always performed regardless of device capability.

## VIII. Evaluation

We demonstrate the system end-to-end across three scenarios: an L2 login (first-ever login, password and TOTP); an L1 login (returning user from a known IP, password only); and an L3 login (elevated risk, all three factors). We additionally verify end-to-end tunnel connectivity and RBA score correctness for each tier. All scenarios start from a clean run of `scripts/setup.py` on a fresh checkout.

### A. Stack Setup

**Prerequisites.** The host must have `docker`, `docker compose` (v2), and `wireguard-tools` installed, and the current user must be in the `docker` group:

```
sudo apt install docker.io docker-compose-v2 wireguard-tools
sudo usermod -aG docker $USER && newgrp docker
sudo modprobe wireguard    # verify kernel module is
     available
```

No Python dependencies are required on the host; every Python process runs inside a container.

**Setup.** `scripts/setup.py` is idempotent: it generates all secrets and the server WireGuard keypair, builds the Docker images, starts the three server-side containers (`vpn-idp`, `vpn-helpdesk`, `vpn-srvauthn`), and enrolls the test user `alice` with password and TOTP. The optional client container (`vpn-client`) is brought up separately when running the authentication scenarios. Figs. 9 and 10 show a clean run and Fig. 11 confirms the containers are up with the expected port bindings.

```
rita@rita:~/Desktop/MSc/2_Semestre/IAA/Projetos/WireGuard-Authn-IAA$ python3 scripts/setup.py

  WireGuard-Authn — Stack Setup


Step 1: Check prerequisites

  ✓  docker found
  ✓  wg found
  ✓  docker compose v2 found
  ✓  Docker daemon reachable

Step 2: Environment file (.env)

  ✓  .env written with fresh secrets (incl. IAA_MASTER_KEY for §4.6)

Step 3: WireGuard server keypair

  ✓  WireGuard keypair generated
  ✓  Server public key: wT0UU2zURjq6mqVaHDXTnGYNtliRQaExglXxQLzrihc=
  ✓  wg0.conf [Interface] PrivateKey already in sync
  ✓  SERVER_PUBKEY written to .env

Step 4: SrvAuthn config (server/srvauthn.conf)

  ✓  server/srvauthn.conf written

Step 4b: Client base config (client/base.conf)

  ✓  client/base.conf written

Step 4c: Runtime data directory (data/)

  ✓  data/ ready for Docker bind mount

Step 5: Build and start Docker stack

  →  Building images (first time takes 1-2 minutes)...
```

Fig. 9. `setup.py`: config and key generation (steps 1–4).

```
[+] up 7/7
 ✔ Image wireguard-authn-iaa-helpdesk  Built
 ✔ Image wireguard-authn-iaa-srvauthn  Built
 ✔ Image wireguard-authn-iaa-idp       Built
 ✔ Network wireguard-authn-iaa_default Created
 ✔ Container vpn-idp                   Started
 ✔ Container vpn-helpdesk              Started
 ✔ Container vpn-srvauthn              Started
  →  Waiting for services to be ready...
  ✓  vpn-idp      ready on :8080
  ✓  vpn-helpdesk ready on :8081
  ✓  vpn-srvauthn ready on :9999 (Noise-IK)
  ✓  wg0 up and listening on UDP 51820

Step 6: Enrol test user 'alice'

  ✓  User 'alice' created
  ✓  Password set
  ✓  TOTP secret generated
  ✓  TOTP confirmed (code=822739)
  ✓  'alice' fully enrolled: password ✓  TOTP ✓

  Setup complete!


  Services:
    IdP       →  http://localhost:8080
    Helpdesk  →  http://localhost:8081
    SrvAuthn  →  port 9999 (Noise-IK)
    WireGuard →  UDP 51820

  Test user:  alice  /  TestPass123
```

Fig. 10. `setup.py`: stack up and user enrolled (steps 5–6).

```
rita@rita:~/Desktop/MSc/2_Semestre/IAA/Projetos/WireGuard-Authn-IAA$ docker ps
CONTAINER ID   IMAGE                      COMMAND                  CREATED         STATUS         PORTS                                       NAMES
fc4285d479e7   wireguard-authn-iaa-helpdesk   "python -m helpdesk._…"   3 minutes ago   Up 3 minutes                                               vpn-helpdesk
12cf6e854d74   wireguard-authn-iaa-srvauthn   "/app/server/entrypo…"   3 minutes ago   Up 3 minutes                                               vpn-srvauthn
a8d49d03101a   wireguard-authn-iaa-idp        "python -m idp.app"      3 minutes ago   Up 3 minutes   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp   vpn-idp
```

Fig. 11. Running containers after setup, with port bindings.

### B. Helpdesk Enrolment

The helpdesk at `http://localhost:8081` manages all three factors per user (Figs. 12 and 13). The TOTP QR code and FIDO2 registration prompts are shown in Figs. 3 and 5 (Section IV). For the FIDO2 enrolment, the virtual FIDO device (`sudo python3 virtual_fido/src-fido/virtual_fido.py`) must be running.

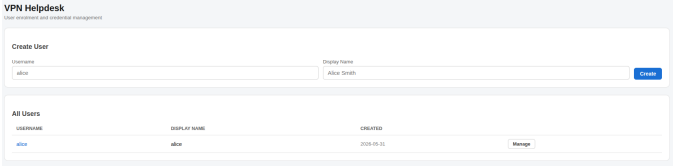



Fig. 12. Helpdesk user list.

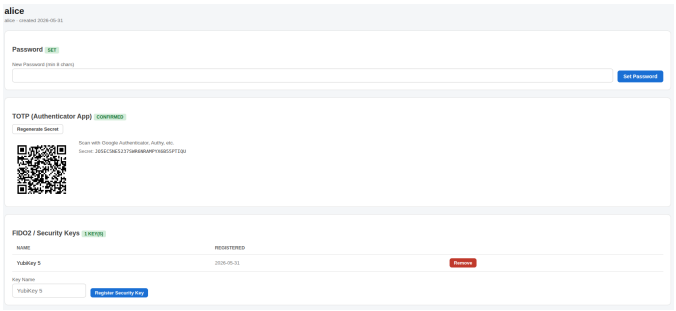



Fig. 13. Per-user panel, all factors enrolled.

### C. L2 Authentication: Password and TOTP

To reproduce, bring up the optional client container and run `userauthn.py` inside it:

```
docker compose --profile client up -d --build client
docker exec vpn-client python3 -u /workspace/client/
    userauthn.py \
  --interface wg0 --username alice \
  --config /workspace/client/base.conf --proxy-host 0.0.0.0
```

A first login from a clean state scores 0.30 (the first-ever-login signal), placing `alice` at tier L2: password and TOTP. `UserAuthn` generates the PSK, opens the Noise-IK tunnel, and prints the device code (Fig. 14).

```
Open this URL in your browser to authenticate:

http://localhost:18080/oauth/device?code=QJLR-NGBP

Your code: QJLR-NGBP
```

Fig. 14. `UserAuthn` terminal at authentication start.

The browser opens the OAuth device page with the code pre-filled (Fig. 15).

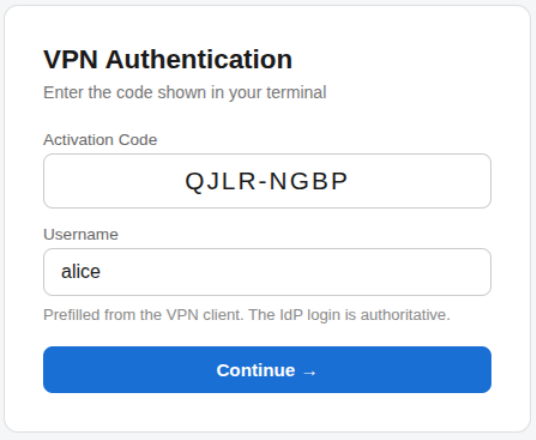



Fig. 15. OAuth device page: code pre-filled via redirect URI.

The user completes the password factor (Fig. 2) and the TOTP factor (Fig. 4). On success the browser shows the completion page (Fig. 16). This page and the terminal banner below are identical for every tier (only the factor sequence leading up to them differs), so we show them once here and refer back to them in the L1 and L3 scenarios.

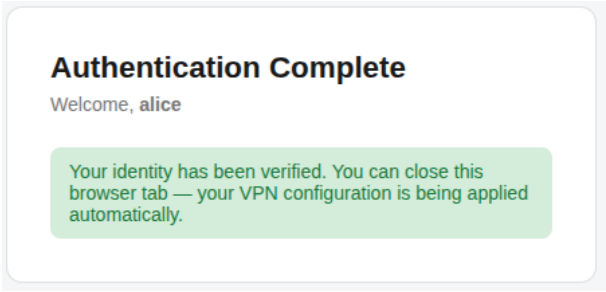



Fig. 16. Browser "Authentication Complete" page shown on a successful login (identical across all tiers).

The terminal then shows the leased tunnel IP, confirming the WireGuard interface is up (Fig. 17).

```
            VPN Connected!

User:      alice
Interface: wg0
IP:        10.10.10.2
```

Fig. 17. Client terminal "VPN Connected" banner with the leased tunnel IP (identical across all tiers).

### D. L1 Authentication: Password Only

After the L2 login above, `alice`'s IP is recorded in `rba_ip_history`. A second login from the same machine during normal hours with no recent failures scores 0.00 (no signal fires), placing `alice` at tier L1: password only. To reproduce: immediately after the L2 session, run `python3 scripts/teardown.py --session` to clear the WireGuard peer and session tables while preserving `rba_ip_history`, then run `userauthn.py` again. The browser shows only the password factor (Fig. 18); the TOTP step does not appear.

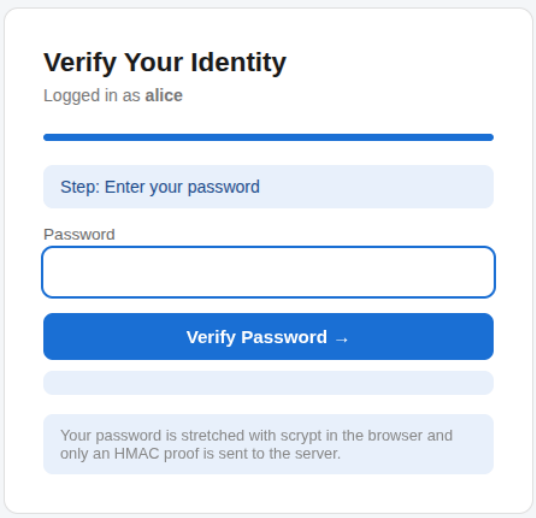



Fig. 18. Password-only factor page during an L1 login.

After the single factor succeeds, the browser shows the same completion page (Fig. 16) and the tunnel comes up with the same "VPN Connected" banner (Fig. 17) as in the L2 case.

### E. L3 Authentication: Password, TOTP, and FIDO2

The FIDO2 factor is required at tier L3, reached when the risk score is at least 0.60. To reproduce: reset the DB auth tables (`teardown.py --reset`), ensure `alice` has a FIDO2 credential enrolled, then make *four failed password attempts in separate sessions* before the real login (the risk score is computed at session start, so only failures from prior sessions count toward it). Four failures add $4 \times 0.10 = 0.40$ to the first-login baseline of 0.30, giving a score of $0.70 \geq 0.60$. The virtual FIDO device must be running. After password and TOTP pass, the browser presents the FIDO2 assertion dialog (Fig. 19).

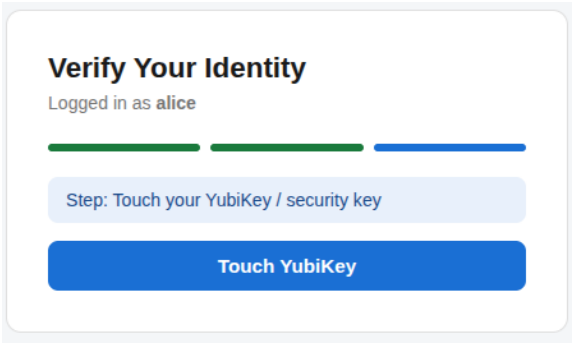



Fig. 19. FIDO2 factor step during an L3 VPN login.

Once all three factors pass, the browser shows the same completion page (Fig. 16) and the terminal prints the same "VPN Connected" banner (Fig. 17) as in the previous scenarios.

### F. Server-Side Verification

After any successful login, `wg show wg0` inside the `vpn-srvauthn` container lists the provisioned peer with its tunnel IP, pre-shared key, and most recent handshake timestamp (Fig. 20).

```
rita@rita:~/Desktop/MSc/2_Semestre/IAA/Projetos/WireGuard-Authn-IAA$ docker exec vpn-srvauthn wg show wg0
interface: wg0
  public key: wT0UU2zURjq6mqVaHDXTnGYNtliRQaExglXxQLzrihc=
  private key: (hidden)
  listening port: 51820

peer: 5yshGkZw9b2WFqvEYrPFvhgBjprP1nNJLYRfoHXh3E4=
  preshared key: (hidden)
  endpoint: 172.24.0.3:45199
  allowed ips: 10.10.10.2/32
  latest handshake: 50 seconds ago
  transfer: 212 B received, 156 B sent
  persistent keepalive: every 25 seconds

peer: Z+PvSk2LfmBUZJ0sRmd6sFoLZOSyDlqYgm4F1bdcFGQ=
  preshared key: (hidden)
  endpoint: 172.24.0.3:48334
  allowed ips: (none)
  latest handshake: 5 minutes, 2 seconds ago
  transfer: 1.58 KiB received, 4.17 KiB sent
  persistent keepalive: every 25 seconds
```

Fig. 20. Server-side `wg show wg0` after provisioning.

A `ping` from the client container to the server tunnel address `10.10.10.1` confirms that the tunnel carries traffic end-to-end (Fig. 21).

```
rita@rita:~/Desktop/MSc/2_Semestre/IAA/Projetos/WireGuard-Authn-IAA$ docker exec vpn-client ping -c 3 10.10.10.1
PING 10.10.10.1 (10.10.10.1) 56(84) bytes of data.
64 bytes from 10.10.10.1: icmp_seq=1 ttl=64 time=0.189 ms
64 bytes from 10.10.10.1: icmp_seq=2 ttl=64 time=0.419 ms
64 bytes from 10.10.10.1: icmp_seq=3 ttl=64 time=0.313 ms

--- 10.10.10.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2079ms
rtt min/avg/max/mdev = 0.189/0.307/0.419/0.093 ms
```

Fig. 21. End-to-end ping through the WireGuard tunnel.

### G. RBA Score Verification

The RBA engine logs the computed score and active signals on every login. A first-ever login fires only the `first_login` signal (score 0.30, tier L2). A returning user from a known IP during normal hours fires no signals (score 0.00, tier L1). A first login preceded by four failed attempts in previous sessions accumulates $0.30 + 4 \times 0.10 = 0.70$, exceeding the L3 threshold of 0.60. Fig. 22 shows the IdP log output for all three scenarios together.

```
2026-05-31 19:48:03,976 [rba] INFO user=alice score=0.30 signals=['first_login'] tier=L2
2026-05-31 19:48:42,043 [rba] INFO user=alice score=0.00 signals=[] tier=L1
2026-05-31 19:50:46,592 [rba] INFO user=alice score=0.70 signals=['first_login', 'recent_failures'] tier=L3
```

Fig. 22. IdP log output confirming the RBA score and tier for each scenario.

### H. Discussion

The Noise-IK handshake and WireGuard bring-up complete in well under a second on loopback; `wg-quick up` takes roughly 200 ms. The dominant wall-clock cost is the user completing MFA in the browser. The OAuth polling interval is 5 s, so the maximum additional delay between "MFA done" and "tunnel active" is one poll cycle plus the bring-up time, well within acceptable limits for a VPN login.

The risk score is deterministic given a fixed DB state, so every scenario is fully reproducible. The RBA log output

provides direct evidence that the score and tier assignment match the signal weights described in Section VI, with no manual inspection of internal state required.

## IX. Conclusion

We built a complete user-authentication layer for WireGuard: a `UserAuthn` client and `SrvAuthn` gateway communicating through a hand-implemented Noise-IK tunnel anchored on the gateway's WireGuard public key; an OAuth 2.0 Device Authorization Grant IdP that hosts three MFA factors (password challenge-response, TOTP, FIDO2); a custom C PAM module that selects the factor sequence from a runtime risk score; an RBA engine driven by IP history, ASN changes, recent failures, and time-of-day; and a loopback-only helpdesk for enrolment. All twelve WireGuard parameters (six per side) are produced and applied dynamically on each login, as shown in Tables III and IV.

Beyond the core functionality, we implemented five additional security hardening measures (Section VII). The repository ships with `scripts/setup.py` and `scripts/teardown.py` and all three evaluation scenarios are reproducible as described in Section VIII.

### A. Features Not Implemented

No mandatory feature is left unimplemented. Two deliberate implementation choices deviate slightly from the most literal reading of the spec:

- **Server tunnel IP in `srvauthn.conf`:** The spec asks the server base config to carry the server's own tunnel IP as an explicit parameter. We set it in `wg0.conf` instead, which is the standard location for a `wg-quick` interface address. Placing it in `srvauthn.conf` as well would duplicate the value across two files that must be kept in sync without adding any functionality, since `wg-quick` reads only `wg0.conf`. The net effect is identical (Section III-E).
- **RBA ASN signal:** The ASN signal is fully implemented using a local db-ip ASN database (MMDB format) mounted into the IdP container. Because such databases require a separate download, the engine is designed to degrade gracefully: if no MMDB file is present it disables that single signal and continues on the remaining four. In the provided Docker setup the file is included and the signal is active.

### B. Author Contributions

TABLE VII
Authors and contribution percentages.

| NMEC | Name | Contribution |
|---|---|---|
| 114220 | Ana Rita Silva | 25% |
| 43483 | Carlos Coelho | 25% |
| 108545 | Miguel Reis | 25% |
| 103234 | Paulo Pinto | 25% |

## References


[1] J. A. Donenfeld, "WireGuard: Next generation kernel network tunnel," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. San Diego, CA, USA: Internet Society, 2017.

[2] W. Denniss, J. Bradley, M. Jones, and H. Tschofenig, "OAuth 2.0 Device Authorization Grant," RFC 8628, 2019.

[3] T. Perrin, "The Noise protocol framework," noiseprotocol.org, Tech. Rep., 2018, revision 34. https://noiseprotocol.org/noise.html.

[4] A. Langley, M. Hamburg, and S. Turner, "Elliptic curves for security," RFC 7748, 2016.

[5] Y. Nir and A. Langley, "ChaCha20 and Poly1305 for IETF protocols," RFC 8439, 2018.

[6] J.-P. Aumasson, S. Neves, Z. Wilcox-O'Hearn, and C. Winnerlein, "BLAKE2: Simpler, smaller, fast as MD5," in *Applied Cryptography and Network Security (ACNS 2013)*. Springer, 2013.

[7] C. Percival and S. Josefsson, "The scrypt password-based key derivation function," RFC 7914, 2016.

[8] C. Percival, "Stronger key derivation via sequential memory-hard functions," in *BSDCan 2009*, 2009.

[9] D. M'Raihi, S. Machani, M. Pei, and J. Rydell, "TOTP: Time-based one-time password algorithm," RFC 6238, 2011.

## APPENDIX A
## AUTHENTICATION PROTOCOL SEQUENCE DIAGRAM

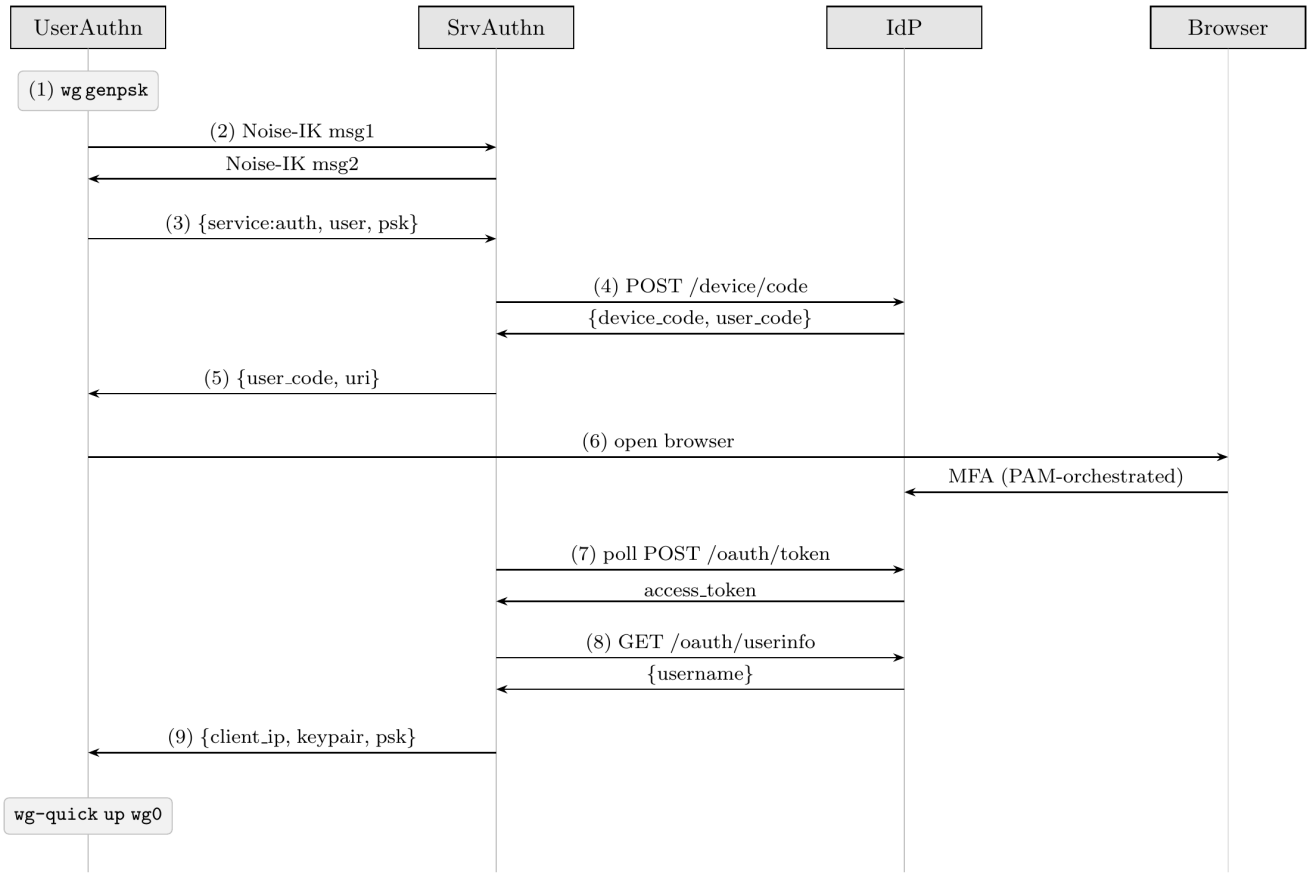



Fig. 23. Sequence diagram of the nine-step authentication protocol. Step numbers correspond to the enumerated list in Section III.